

Fase 1: Fundamentos de SOC (Security Operations Center)

Instructor: Instructor SOC — Maestro Senior SOC **Formato:** Teoría expandida + Laboratorio práctico en terminal (Linux/Parrot OS) — Comandos numerados con explicación inline

Índice

1. ¿Qué es un SOC? Estructura, Tiers y Ciclo de Vida del Analista
2. Logs — La Materia Prima del Analista SOC
3. Fundamentos de Redes para el Analista SOC
4. Sistemas Operativos desde la Óptica SOC
5. Cyber Kill Chain y MITRE ATT&CK
6. Tipos de Malware y su Comportamiento
7. Criptografía Esencial para el Analista SOC
8. Threat Intelligence, IOCs y OSINT
9. Ciclo de Vida de Respuesta a Incidentes (NIST SP 800-61)
10. Windows Event Logs — Cimientos del SOC en Entornos Windows

Tema 1: ¿Qué es un SOC? Estructura, Tiers y Ciclo de Vida del Analista

Concepto

Un **SOC (Security Operations Center)** es la torre de control de una organización frente a las amenazas cibernéticas — el equivalente digital de una sala de emergencias, donde las alertas son “pacientes” que requieren triage.

La estructura se organiza en **Tiers**:

- **Tier 1:** Monitoreo en tiempo real y triage inicial de alertas usando playbooks.
- **Tier 2:** Investigación profunda, correlación de eventos, determinación del alcance real del incidente.
- **Tier 3:** Threat hunters e ingenieros de detección que manejan amenazas sofisticadas y mejoran reglas de detección.

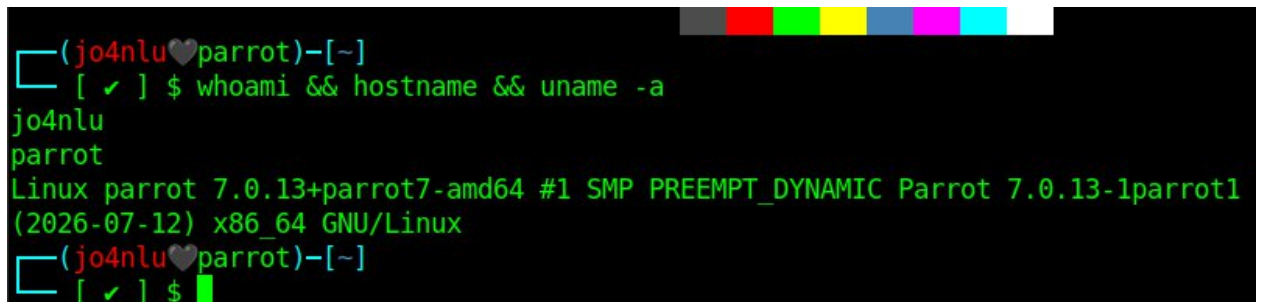
Caso real: La brecha de **Target (2013)** expuso 40 millones de tarjetas de crédito. El sistema de detección FireEye *sí generó la alerta*, pero el equipo la desestimó en medio del ruido diario. Lección: la tecnología detecta, el proceso y el criterio humano deciden la respuesta.

Ciclo operativo: Monitoreo → Detección → Triage → Investigación → Contención → Erradicación → Recuperación → Lecciones aprendidas (basado en NIST SP 800-61).

Laboratorio

Comando 1:

whoami && hostname && uname -a



```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ whoami && hostname && uname -a
jo4nlu
parrot
Linux parrot 7.0.13+parrot7-amd64 #1 SMP PREEMPT_DYNAMIC Parrot 7.0.13-1parrot1
(2026-07-12) x86_64 GNU/Linux
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: confirma tu identidad de usuario actual, el nombre del host y la información del kernel/arquitectura del sistema — el primer reflejo de cualquier analista al sentarse frente a un sistema, propio o bajo investigación: “¿quién soy y en qué máquina estoy?”.

Comando 2:

`ls -la /var/log/ | head -20`

```
(jo4nlu♥parrot)-[~]  
[✓] $ ls -la /var/log/ | head -20  
total 99640  
drwxr-xr-x 1 root      root      1276 jul 19 18:49 .  
drwxr-xr-x 1 root      root       130 jul 18 22:43 ..  
drwxr-xr-x 1 root      root        10 jul  2 10:15 account  
-rw-r--r-- 1 root      root    10838 jul 17 19:33 alternatives.log  
-rw-r--r-- 1 root      root   24482 jun 27 14:39 alternatives.log  
.1  
drwxr-x--- 1 root      adm        638 jul 17 20:15 apache2  
drwxr-xr-x 1 root      root         0 abr 10  2025 apparmor  
drwxr-xr-x 1 root      root       118 jul 17 19:33 apt  
drwxr-xr-x 1 root      root       286 jul 19 18:46 atop  
-rw-r--r-- 1 root      root   201005 jul 18 22:40 auth.log  
-rw-r--r-- 1 root      root   633166 jul 15 14:17 auth.log.1  
drwxr-xr-x 1 _bloodhound _bloodhound 0 abr 15 06:32 bloodhound  
-rw----- 1 root      root   35794 jul 19 18:46 boot.log  
-rw----- 1 root      root   35231 jul 17 20:15 boot.log.1  
-rw----- 1 root      root   52529 jul 16 16:12 boot.log.2  
-rw----- 1 root      root  104772 jul 15 14:23 boot.log.3  
-rw----- 1 root      root   52265 jul 13 00:11 boot.log.4  
-rw----- 1 root      root   83414 jul 11 14:41 boot.log.5  
-rw----- 1 root      root   30617 jul 10 00:02 boot.log.6
```

Qué hace: lista el contenido del directorio central de logs en Linux con detalles de permisos, propietario y fecha de modificación — te da un primer mapa de qué fuentes de log existen en este sistema antes de empezar a investigarlas una por una.

Comando 3:

`last -a | head -15`

```
(jo4nlu♥parrot)-[~]  
[✓] $ last -a | head -15  
jo4nlu    tty7          Sun Jul 19 18:49 - still logged in      :0  
lightdm   tty7          Sun Jul 19 18:46 - 18:46 (00:02)      :0  
reboot    system boot  Sun Jul 19 18:46 - still running      7.0.13+parrot7-amd64  
jo4nlu    tty7          Sat Jul 18 11:14 - 23:34 (12:20)      :0  
lightdm   tty7          Sat Jul 18 11:13 - 11:14 (00:00)      :0  
reboot    system boot  Sat Jul 18 11:13 - 23:34 (12:21)      7.0.13+parrot7-amd64  
jo4nlu    tty7          Fri Jul 17 19:25 - 23:14 (03:49)      :0  
lightdm   tty7          Fri Jul 17 19:24 - 19:25 (00:00)      :0  
reboot    system boot  Fri Jul 17 19:24 - 23:14 (03:49)      7.0.13+parrot7-amd64  
jo4nlu    tty7          Thu Jul 16 22:02 - 23:59 (01:56)      :0  
lightdm   tty7          Thu Jul 16 22:01 - 22:02 (00:00)      :0  
reboot    system boot  Thu Jul 16 22:01 - 23:59 (01:57)      7.0.13+parrot7-amd64  
jo4nlu    tty7          Thu Jul 16 16:06 - crash              :0  
lightdm   tty7          Thu Jul 16 16:02 - 16:06 (00:03)      :0  
reboot    system boot  Thu Jul 16 16:02 - crash              7.0.13+parrot7-amd64
```

Qué hace: muestra el historial de inicios de sesión recientes, incluyendo desde qué host se conectó cada usuario — la primera pregunta de triage en cualquier investigación: “¿quién ha entrado aquí últimamente?”.

Tema 2: Logs — La Materia Prima del Analista SOC

Concepto

Un **log** es un registro cronológico de un evento en un sistema. Regla no escrita: “*si no está en el log, no pasó — o peor, pasó y no lo viste*”.

Clasificación por fuente: sistema operativo, aplicación, red, seguridad específica. En Linux conviven **syslog** (/var/log/) con **journald** (systemd, consultado vía journalctl).

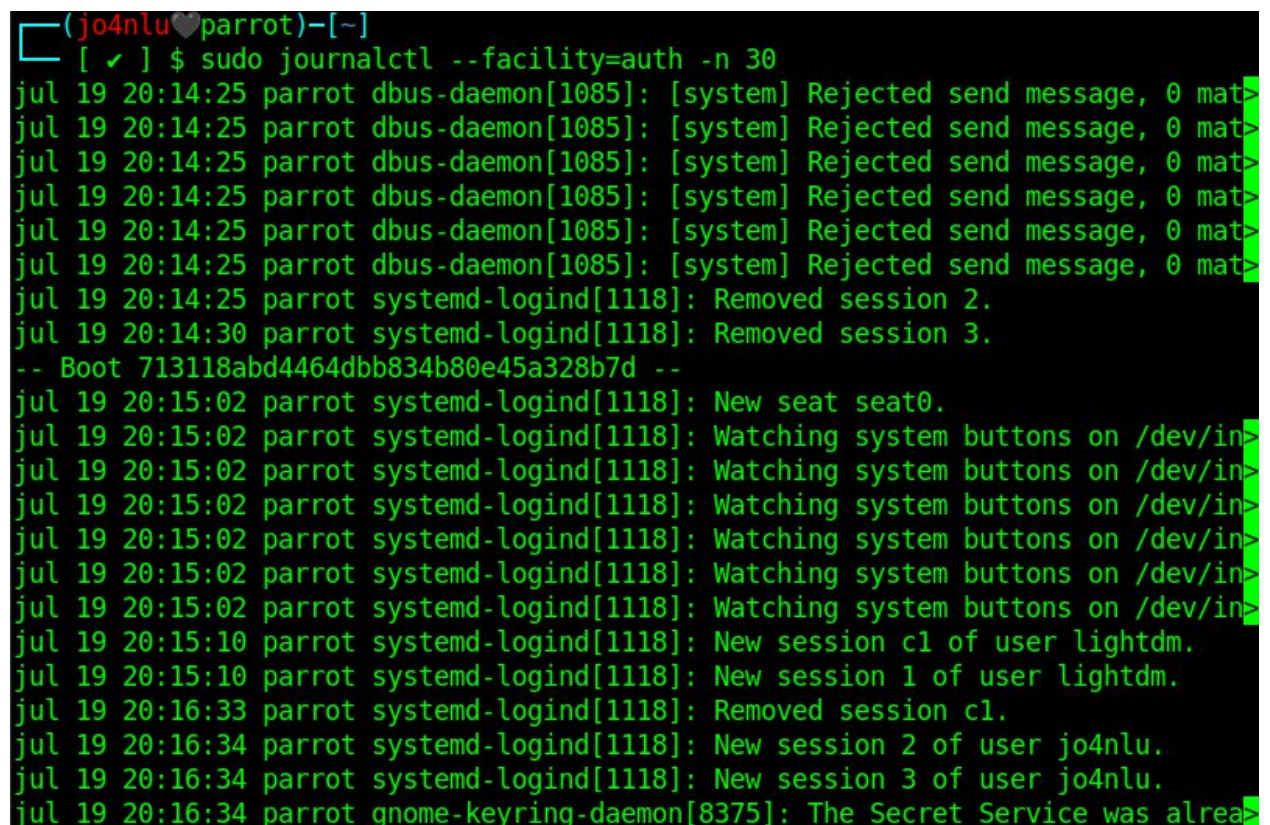
Caso real: El ataque a la cadena de suministro de **SolarWinds (2020)** insertó la puerta trasera SUNBURST evadiendo generación de logs sospechosos (nombres de procesos legítimos, activación retardada, tráfico C2 disfrazado). La reconstrucción forense dependió enteramente de logs de DNS, autenticación y red retenidos.

Tríada de calidad del log: Cobertura, Retención e Integridad (forwarding a servidor centralizado para evitar manipulación por el atacante).

Laboratorio

Comando 1:

`sudo tail -n 30 /var/log/auth.log`



```
(jo4nlu@parrot)-[~]  
[ ✓ ] $ sudo journalctl --facility=auth -n 30  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot dbus-daemon[1085]: [system] Rejected send message, 0 mat>  
jul 19 20:14:25 parrot systemd-logind[1118]: Removed session 2.  
jul 19 20:14:30 parrot systemd-logind[1118]: Removed session 3.  
-- Boot 713118abd4464dbb834b80e45a328b7d --  
jul 19 20:15:02 parrot systemd-logind[1118]: New seat seat0.  
jul 19 20:15:02 parrot systemd-logind[1118]: Watching system buttons on /dev/in>  
jul 19 20:15:02 parrot systemd-logind[1118]: Watching system buttons on /dev/in>  
jul 19 20:15:02 parrot systemd-logind[1118]: Watching system buttons on /dev/in>  
jul 19 20:15:02 parrot systemd-logind[1118]: Watching system buttons on /dev/in>  
jul 19 20:15:02 parrot systemd-logind[1118]: Watching system buttons on /dev/in>  
jul 19 20:15:10 parrot systemd-logind[1118]: New session c1 of user lightdm.  
jul 19 20:15:10 parrot systemd-logind[1118]: New session 1 of user lightdm.  
jul 19 20:16:33 parrot systemd-logind[1118]: Removed session c1.  
jul 19 20:16:34 parrot systemd-logind[1118]: New session 2 of user jo4nlu.  
jul 19 20:16:34 parrot systemd-logind[1118]: New session 3 of user jo4nlu.  
jul 19 20:16:34 parrot gnome-keyring-daemon[8375]: The Secret Service was alrea>
```

Qué hace: muestra las últimas 30 líneas del log de autenticación — logins, uso de sudo, intentos SSH — el archivo más consultado por un analista Tier 1 en su turno diario para detectar accesos sospechosos.

Comando 2:

```
journalctl -p err -b --no-pager | head -20
```

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ journalctl -p err -b --no-pager | head -20
Hint: You are currently not seeing messages from other users and the system.
      Users in groups 'adm', 'systemd-journal' can see all messages.
      Pass -q to turn off this notice.
jul 19 20:16:35 parrot obexd[8841]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
jul 19 20:16:35 parrot obexd[8841]: Unable to acquire registry: Error calling StartServiceByName for org.gnome.evolution.dataserver.Sources5: Unit evolution-source-registry.service not found.
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: filtra el journal de systemd mostrando solo mensajes de severidad **error o superior** desde el último arranque del sistema (-b) — practica la habilidad esencial de filtrar por severidad para separar ruido de señal.

Comando 3:

```
sudo find /var/log -type f -mmin -60 -exec ls -lh {} \;
```

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ sudo find /var/log -type f -mmin -60 -exec ls -lh {} \;
-rw-r----- 1 root adm 1,4K jul 19 19:26 /var/log/apache2/error.log.1
-rw-r----- 1 root adm 644 jul 19 20:15 /var/log/apache2/error.log
-rw-r-----+ 1 root systemd-journal 40M jul 19 20:18 /var/log/journal/2cc37b94e7034f40a81145053b73190b/systemd-journal
-rw-r-----+ 1 root systemd-journal 8,0M jul 19 20:19 /var/log/journal/2cc37b94e7034f40a81145053b73190b/user-1000.journal
-rw-r--r-- 1 root root 33K jul 19 20:15 /var/log/sysstat/sa19
-rw-rw-r-- 1 root utmp 81K jul 19 20:16 /var/log/wtmp
-rw----- 1 root root 17K jul 19 20:15 /var/log/boot.log
-rw----- 1 root root 5,9K jul 19 20:14 /var/log/lightdm/lightdm.log.old
-rw----- 1 root root 4,9K jul 19 20:16 /var/log/lightdm/lightdm.log
-rw----- 1 root root 996 jul 19 20:14 /var/log/lightdm/x-0.log.old
-rw----- 1 root root 937 jul 19 20:15 /var/log/lightdm/x-0.log
-rw----- 1 root root 7,5K jul 19 20:16 /var/log/lightdm/seat0-greeter.log
-rw-r--r-- 1 root root 16K jul 19 20:16 /var/log/wtmp.db
-rw-r----- 1 proxy proxy 54K jul 19 19:26 /var/log/squid/cache.log.1
-rw-r----- 1 proxy proxy 10K jul 19 20:15 /var/log/squid/cache.log
-rw-r----- 1 proxy proxy 0 jul 19 19:26 /var/log/squid/access.log
-rw-r--r-- 1 root root 1,1M jul 19 20:15 /var/log/atop/atop_20260719
-rw----- 1 root root 35K jul 19 19:26 /var/log/boot.log.1
-rw-r----- 1 root adm 3,5K jul 19 20:15 /var/log/fail2ban.log
-rw-r--r-- 1 root root 0 jul 19 19:26 /var/log/syslog
```

Qué hace: identifica qué archivos de log se han modificado en los últimos 60 minutos — útil en una investigación en vivo para saber rápidamente qué fuentes están “hablando” activamente ahora mismo.

Tema 3: Fundamentos de Redes para el Analista SOC

Concepto

El modelo **TCP/IP** (4 capas: Acceso a Red, Internet, Transporte, Aplicación) sustenta toda investigación de tráfico. **TCP** (orientado a conexión, three-way handshake: SYN, SYN-ACK, ACK) vs **UDP** (sin garantía de entrega, más rápido).

Puertos: Well-known (0-1023: SSH 22, HTTP 80, HTTPS 443, SMB 445), registered (1024-49151), dinámicos (49152-65535). Memorizarlos es esencial para triage rápido.

Caso real: WannaCry (2017) explotó EternalBlue en el puerto **445 (SMB)**, infectando 200,000+ sistemas en 150 países. Equipos con visibilidad **este-oeste** (tráfico interno, no solo perimetral) detectaron el brote en minutos.

Laboratorio

Comando 1:

ss -tunap

```
(jo4nlu❤parrot)-[~]
[ ✓ ] $ ss -tunap
```

Netid	State	Recv-Q	Send-Q	Peer Address:Port	Local Address:Port
Process					
udp	UNCONN	0	0	0.0.0.0:*	172.24.0.1:123
udp	UNCONN	0	0	0.0.0.0:*	172.21.0.1:123
udp	UNCONN	0	0	0.0.0.0:*	172.20.10.3:123
udp	UNCONN	0	0	0.0.0.0:*	100.72.113.2:123
udp	UNCONN	0	0	0.0.0.0:*	127.0.0.1:123
udp	UNCONN	0	0	0.0.0.0:*	0.0.0.0:123

Qué hace: muestra todas las conexiones TCP/UDP activas de tu máquina en este momento — quién escucha, quién está conectado y a qué puertos. Es la primera herramienta que corre cualquier analista al investigar un host sospechoso.

Comando 2:

`sudo tcpdump -i any -c 20 -n port 53`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ sudo tcpdump -i any -c 20 -n port 53
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
bytes
20:24:44.915703 tailscale0 Out IP 100.72.113.2.41672 > 100.100.100.100.53: 57538
+ A? www.youtube.com. (33)
20:24:44.915714 tailscale0 Out IP 100.72.113.2.41672 > 100.100.100.100.53: 64451
+ AAAA? www.youtube.com. (33)
20:24:44.915926 wlo1 Out IP 172.20.10.3.36861 > 172.20.10.1.53: 57538+ A? www.y
outube.com. (33)
20:24:44.915947 wlo1 Out IP6 fe80::276c:9c36:4e08:6c64.48888 > fe80::6082:46ff:
fe2a:d664.53: 57538+ A? www.youtube.com. (33)
20:24:44.995100 wlo1 In IP 172.20.10.1.53 > 172.20.10.3.36861: 57538 8/0/0 A 1
42.251.154.4, A 142.251.157.4, A 142.251.151.4, A 142.251.153.4, A 142.251.156.4
, A 142.251.155.4, A 142.251.150.4, A 142.251.152.4 (161)
20:24:44.995526 tailscale0 In IP 100.100.100.100.53 > 100.72.113.2.41672: 57538
8/0/0 A 142.251.154.4, A 142.251.157.4, A 142.251.151.4, A 142.251.153.4, A 142
```

Qué hace: captura en vivo 20 paquetes de tráfico DNS (puerto 53) — practica filtrar tráfico específico entre el ruido general de la interfaz de red, la base de todo análisis posterior con Wireshark/tcpdump.

Comando 3:

`cat /etc/services | grep -E "^(ssh|http|https|ftp|smb|rdp|telnet)\s"`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ cat /etc/services | grep -E "^(ssh|http|https|ftp|smb|rdp|telnet)\s"

ftp          21/tcp
ssh          22/tcp      # SSH Remote Login Protocol
telnet       23/tcp
http         80/tcp      www         # WorldWideWeb HTTP
https        443/tcp     # http protocol over TLS/SSL
https        443/udp     # HTTP/3

(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: muestra cómo el propio sistema operativo mapea nombres de servicios a números de puerto — memorizar esta tabla mental es lo que te permite leer un log de firewall a velocidad de triage sin buscar cada puerto en Google.

Tema 4: Sistemas Operativos desde la Óptica SOC

Concepto

Todo ataque eventualmente se ejecuta en un sistema operativo. Un **proceso** tiene PID, PPID y usuario propietario — su **árbol de ejecución** cuenta una historia (ej. winword.exe → powershell.exe → red es una firma clásica de macro malicioso).

Privilegios: en Linux, root (UID 0) vs privilegios delegados (sudo, SUID/SGID, grupos como docker). La **escalación de privilegios** es casi universal: el atacante entra con bajo privilegio y busca activamente escalar.

Caso real: El ataque a **Uber (2022)** combinó fatiga de MFA para obtener acceso inicial de bajo privilegio, seguido del hallazgo de credenciales hardcodeadas en un script de PowerShell en un recurso compartido — escalando a control administrativo total en horas.

Laboratorio

Comando 1:

`ps -eo pid,ppid,user,cmd --forest | head -30`

```
(jo4nlu❤️parrot)-[~]
[ ✓ ] $ ps -eo pid,ppid,user,cmd --forest | head -30
  PID    PPID  USER      CMD
    2         0  root      [kthreadd]
    3         2  root      \_ [pool_workqueue_release]
    4         2  root      \_ [kworker/R-rcu_gp]
    5         2  root      \_ [kworker/R-sync_wq]
    6         2  root      \_ [kworker/R-kvfree_rcu_reclaim]
    7         2  root      \_ [kworker/R-slub_flushwq]
    8         2  root      \_ [kworker/R-netns]
   10         2  root      \_ [kworker/0:0H-i915_cleanup]
   11         2  root      \_ [kworker/0:1-mm_percpu_wq]
   13         2  root      \_ [kworker/R-mm_percpu_wq]
   14         2  root      \_ [ksoftirqd/0]
   15         2  root      \_ [rcu_preempt]
   16         2  root      \_ [rcu_exp_par_gp_kthread_worker/0]
   17         2  root      \_ [rcu_exp_gp_kthread_worker]
   18         2  root      \_ [migration/0]
   19         2  root      \_ [kprobe-optimizer]
   20         2  root      \_ [idle_inject/0]
   21         2  root      \_ [cpuhp/0]
   22         2  root      \_ [cpuhp/1]
   23         2  root      \_ [idle_inject/1]
   24         2  root      \_ [migration/1]
   25         2  root      \_ [ksoftirqd/1]
   27         2  root      \_ [kworker/1:0H-kblockd]
```

Qué hace: muestra el árbol de procesos completo con jerarquía padre-hijo — exactamente la vista que usarías para detectar una cadena anómala tipo “documento → shell → red”.

Comando 2:

`sudo find / -perm -4000 -type f 2>/dev/null`

```
(jo4nlu❤parrot)-[~]  
[✓] $ sudo find / -perm -4000 -type f 2>/dev/null  
/opt/VirtualBox/VBoxNetDHCP  
/opt/VirtualBox/VBoxNetNAT  
/opt/VirtualBox/VBoxNetAdpCtl  
/opt/VirtualBox/VBoxVolInfo  
/opt/VirtualBox/VirtualBoxVM  
/opt/VirtualBox/VBoxHeadless  
/opt/brave.com/brave/chrome-sandbox  
/usr/bin/chfn  
/usr/bin/chsh  
/usr/bin/fusermount3  
/usr/bin/gpasswd  
/usr/bin/mount  
/usr/bin/newgidmap  
/usr/bin/newgrp  
/usr/bin/newuidmap  
/usr/bin/passwd  
/usr/bin/pkexec  
/usr/bin/su  
/usr/bin/sudo  
/usr/bin/umount  
/usr/bin/vmware-user-suid-wrapper  
/usr/bin/ntfs-3g  
/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
/usr/lib/mysql/plugin/auth_pam_tool_dir/auth_pam_tool  
/usr/lib/openssh/ssh-keysign  
/usr/lib/polkit-1/polkit-agent-helper-1  
/usr/lib/xorg/Xorg.wrap
```

Qué hace: busca todos los binarios con el bit **SUID** activado en el sistema — la lista de “sospechosos habituales” que cualquier atacante revisa primero al buscar escalación de privilegios (investiga GTFOBins para entender por qué esto es crítico).

Comando 3:

`id && groups && sudo -l`

```
(jo4nlu❤parrot)-[~]  
[✖] $ id && groups && sudo -l  
uid=1000(jo4nlu) gid=1002(jo4nlu) grupos=1002(jo4nlu),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),101(netdev),103(scanner),112(bluetooth),117(lpadmin),131(libvirt),985(ollama),1000(docker),1001(podman)  
jo4nlu dialout cdrom floppy sudo audio dip video plugdev netdev scanner bluetooth lpadmin libvirt ollama docker podman  
Matching Defaults entries for jo4nlu on parrot:  
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty  
User jo4nlu may run the following commands on parrot:  
    (ALL : ALL) ALL  
    (ALL) ALL  
(jo4nlu❤parrot)-[~]  
[✓] $
```

Qué hace: muestra tu identidad completa del sistema — tu UID, tus grupos, y qué comandos puedes ejecutar con sudo — el primer chequeo al investigar cualquier host para entender con qué privilegios estás trabajando.

Tema 5: Cyber Kill Chain y MITRE ATT&CK

Concepto

La **Cyber Kill Chain** (Lockheed Martin, 2011) describe 7 fases lineales: Reconocimiento, Weaponización, Entrega, Explotación, Instalación, C2, Acciones sobre Objetivos. Romper la cadena en cualquier fase temprana detiene el ataque completo.

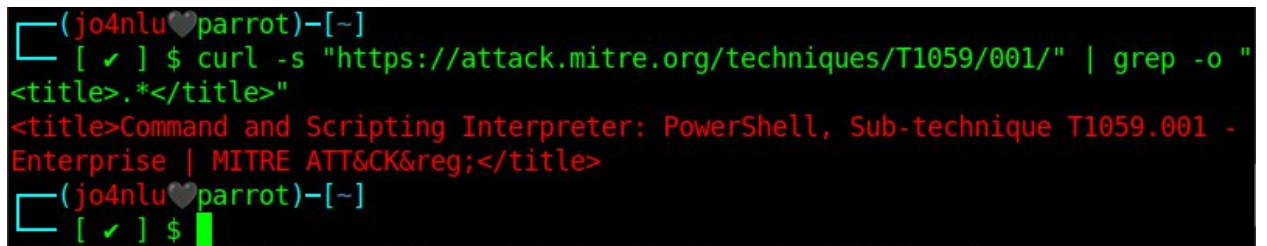
MITRE ATT&CK cataloga Tácticas (el “por qué”), Técnicas (el “cómo”) y Sub-técnicas con ejemplos reales de grupos APT. Lenguaje estandarizado: en vez de “vi algo raro con PowerShell”, se dice “T1059.001 encadenado con T1027 (Obfuscation)”.

Caso real: APT29/Cozy Bear (SolarWinds) fue mapeado con técnicas específicas (T1195.002, T1027, T1090, T1070), permitiendo generalizar defensas contra **TTPs** en vez de solo **IOCs** — “los IOCs son para hoy, los TTPs son para siempre”.

Laboratorio

Comando 1:

```
curl -s "https://attack.mitre.org/techniques/T1059/001/" | grep -o "<title>.*</title>"
```

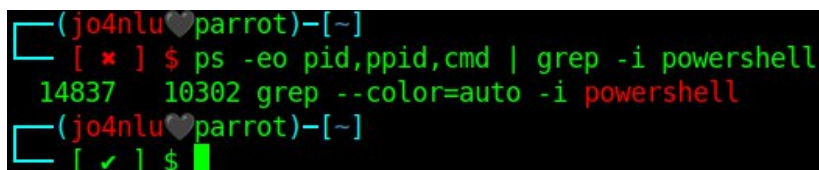


```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ curl -s "https://attack.mitre.org/techniques/T1059/001/" | grep -o "  
<title>.*</title>"  
<title>Command and Scripting Interpreter: PowerShell, Sub-technique T1059.001 -  
Enterprise | MITRE ATT&CK&reg;</title>  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: consulta directamente la página oficial de la técnica **T1059.001 (PowerShell)** en el sitio de MITRE ATT&CK — abre después ese link en tu navegador y explora la matriz completa, la vas a usar constantemente.

Comando 2:

```
ps -eo pid,ppid,cmd | grep -i powershell
```



```
(jo4nlu♥parrot)-[~]  
[ * ] $ ps -eo pid,ppid,cmd | grep -i powershell  
14837  10302 grep --color=auto -i powershell  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: busca procesos relacionados con PowerShell/pwsh corriendo en tu sistema — en Parrot probablemente no encuentres ninguno, y eso también es información valiosa: un baseline limpio.

Comando 3:

history | grep -iE "curl|wget|nc |base64" | tail -20

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ history | grep -iE "curl|wget|nc |base64" | tail -20
2283  curl -s http://10.65.129.167/js/app.js
2284  curl -X POST http://10.65.129.167/api/move -H "Content-Type: applica
tion/json" -d '{"from":"e1", "to":"e8"}'
2286  [ ✓ ] $ curl -s http://10.65.129.167/js/app.js
2321  [ ✓ ] $ curl -s http://10.65.129.167/js/app.js
2355  curl -X POST http://10.65.129.167/api/move -H "Content-Type: applica
tion/json" -d '{"from":"a1", "to":"a8"}'
2469  wget -nc https://dl.winehq.org/wine-builds/winehq.key
2503  sudo wget -O /etc/apt/keyrings/winehq-archive.key https://dl.winehq.org/w
ine-builds/winehq.key
2512  wget https://github.com/Kron4ek/Wine-Builds/releases/download/10.0/wine-1
0.0-amd64.tar.xz
2528  wget https://github.com/Kron4ek/Wine-Builds/releases/download/10.0/wine-1
0.0-wow64.tar.xz
2578  openssl rand -base64 64
2673  curl -fsSL https://ollama.com/install.sh | sh
2981  curl -s "https://attack.mitre.org/techniques/T1059/001/" | grep -o "<titl
e>.*</title>"
2983  history | grep -iE "curl|wget|nc |base64" | tail -20
3006  curl -so wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/w
azuh-agent/wazuh-agent_4.9.2-1_amd64.deb
3034  curl -sO https://raw.githubusercontent.com/wazuh/wazuh-docker/v4.9.0/wazu
```

Qué hace: revisa tu propio historial de bash buscando patrones típicos de **T1105 (Ingress Tool Transfer)** y **T1027 (Obfuscation)** — te enseña a auditar tu propia actividad con la misma mirada sospechosa con la que auditarías la de un atacante.

Tema 6: Tipos de Malware y su Comportamiento

Concepto

Clasificación por comportamiento: **virus**, **gusanos**, **troyanos**, **ransomware** (hoy con doble extorsión), **spyware/keyloggers**, **rootkits**, y **malware fileless** que vive en memoria usando herramientas legítimas (LOLBins) para evadir antivirus basados en firmas.

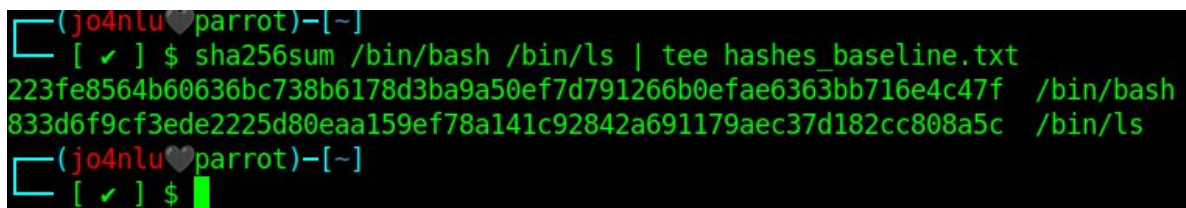
Caso real: NotPetya (2017) se disfrazó de ransomware pero era un **wiper** — destruía datos sin intención de descifrarlos. Se propagó vía EternalBlue combinado con extracción de credenciales en memoria. Costó a Maersk ~300 millones de dólares.

Detección: firmas (hash, rápidas pero evadibles) vs comportamiento/heurística (más robusta contra variantes nuevas).

Laboratorio

Comando 1:

```
sha256sum /bin/bash /bin/ls | tee hashes_baseline.txt
```

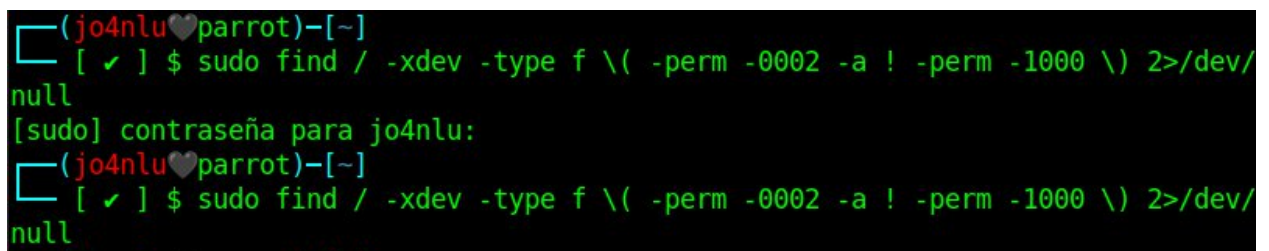


```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ sha256sum /bin/bash /bin/ls | tee hashes_baseline.txt  
223fe8564b60636bc738b6178d3ba9a50ef7d791266b0efae6363bb716e4c47f /bin/bash  
833d6f9cf3ede2225d80eaa159ef78a141c92842a691179aec37d182cc808a5c /bin/ls  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: genera un hash de referencia (baseline) de binarios críticos del sistema — la técnica exacta que usan herramientas de integridad de archivos (FIM) para detectar si un binario del sistema fue reemplazado por una versión troyanizada.

Comando 2:

```
sudo find / -xdev -type f \( -perm -0002 -a ! -perm -1000 \) 2>/dev/null
```



```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ sudo find / -xdev -type f \( -perm -0002 -a ! -perm -1000 \) 2>/dev/  
null  
[sudo] contraseña para jo4nlu:  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ sudo find / -xdev -type f \( -perm -0002 -a ! -perm -1000 \) 2>/dev/  
null
```

Qué hace: busca archivos escribibles por cualquier usuario sin sticky bit — un vector de persistencia clásico donde un atacante deja un script malicioso en una ubicación que cualquier proceso puede sobrescribir.

Comando 3:

`ps aux | awk '{print $11}' | sort | uniq -c | sort -rn | head -15`

```
(jo4nlu❤️parrot)-[~]  
[ ✓ ] $ ps aux | awk '{print $11}' | sort | uniq -c | sort -rn | head -15  
51 /usr/sbin/zabbix_server:  
28 /opt/brave.com/brave/brave  
18 mate-screenshot  
16 /usr/sbin/docker-proxy  
9 php-fpm:  
9 /bin/sh  
7 /app/bin/opt/onlyoffice/desktopeditors/editors_helper  
6 /usr/sbin/apache2  
6 /usr/bin/python3  
6 /usr/bin/containerd-shim-runc-v2  
6 nginx:  
4 /var/ossec/framework/python/bin/python3  
3 /usr/bin/dbus-daemon  
3 s6-supervise  
3 bwrap  
(jo4nlu❤️parrot)-[~]  
[ ✓ ] $
```

Qué hace: genera un censo de qué binarios están corriendo con más frecuencia en tu sistema ahora mismo, estableciendo tu propio baseline de “normalidad” — la base de toda detección de anomalías.

Tema 7: Criptografía Esencial para el Analista SOC

Concepto

Hashing (MD5, SHA-1, SHA-256): función unidireccional para identificar archivos e IOCs.

Cifrado simétrico (AES) vs **asimétrico** (RSA) — HTTPS combina ambos: asimétrico para el handshake, simétrico para la sesión.

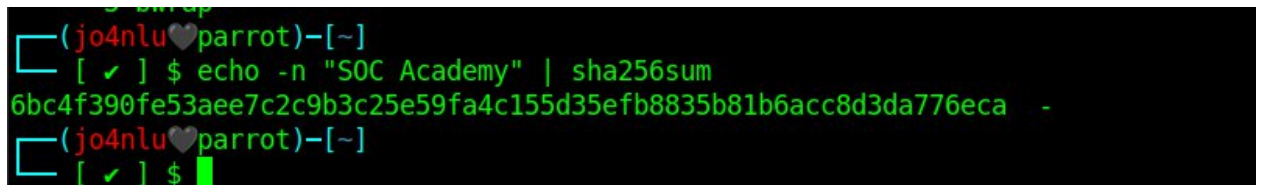
PKI: sistema jerárquico de confianza donde Autoridades Certificadoras (CAs) firman certificados vinculando clave pública con identidad verificada.

Caso real: La brecha de **DigiNotar (2011)** permitió emitir certificados fraudulentos válidos para *google.com*, habilitando ataques man-in-the-middle indetectables contra cientos de miles de usuarios. La compañía quebró en semanas tras perder la confianza de todos los navegadores.

Laboratorio

Comando 1:

```
echo -n "SOC Academy" | sha256sum
```

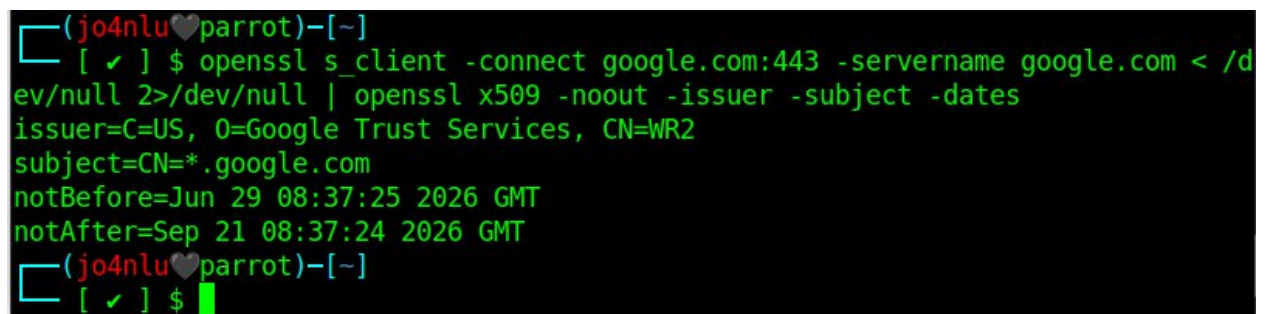


```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ echo -n "SOC Academy" | sha256sum  
6bc4f390fe53aee7c2c9b3c25e59fa4c155d35efb8835b81b6acc8d3da776eca -  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: genera un hash SHA-256 manualmente, para que veas con tus propios ojos cómo un texto arbitrario se convierte en una huella digital de longitud fija — la base de todo IOC basado en hash.

Comando 2:

```
openssl s_client -connect google.com:443 -servername google.com < /dev/null 2>/dev/null  
| openssl x509 -noout -issuer -subject -dates
```



```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ openssl s_client -connect google.com:443 -servername google.com < /dev/null 2>/dev/null | openssl x509 -noout -issuer -subject -dates  
issuer=C=US, O=Google Trust Services, CN=WR2  
subject=CN=*.google.com  
notBefore=Jun 29 08:37:25 2026 GMT  
notAfter=Sep 21 08:37:24 2026 GMT  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: se conecta a un sitio real y extrae los datos de su certificado TLS — quién lo emitió, a quién pertenece, y sus fechas de validez — exactamente lo que revisarías al investigar tráfico HTTPS sospechoso hacia un dominio desconocido.

Comando 3:

```
openssl req -x509 -newkey rsa:2048 -keyout test.key -out test.crt -days 1 -nodes -subj  
"/CN=test-soc-lab"
```

[illegible]

Qué hace: genera tu propio certificado autofirmado, para que entiendas cómo se ve uno sin cadena de confianza hacia una CA reconocida — esto te ayuda a reconocer certificados sospechosos en tráfico de C2 malicioso.

Tema 8: Threat Intelligence, IOCs y OSINT

Concepto

Threat Intelligence (CTI): recolección y análisis de información sobre amenazas para tomar mejores decisiones defensivas. **IOCs:** hashes, IPs, dominios/URLs, correos.

Pirámide del Dolor (David Bianco): ordena indicadores por costo para el atacante al ser detectados — hashes (dolor mínimo) hasta TTPs (dolor máximo, cima de la pirámide).

Herramientas OSINT: VirusTotal, AbuseIPDB, Shodan, MISP.

Caso real: El desmantelamiento parcial de **LockBit** (“Operation Cronos”, 2024) se apoyó en años de inteligencia correlacionando infraestructura entre incidentes atribuidos al grupo.

Laboratorio

Comando 1:

```
curl -s "https://www.virustotal.com/api/v3/ip_addresses/8.8.8.8" -H "accept: application/json" | head -c 500
```

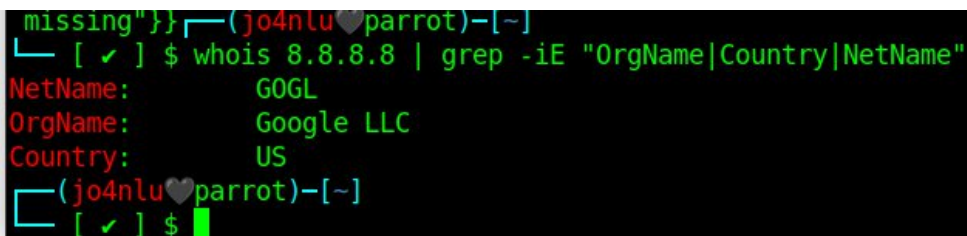


```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ curl -s "https://www.virustotal.com/api/v3/ip_addresses/8.8.8.8" -H  
"accept: application/json" | head -c 500  
{"error": {"code": "AuthenticationRequiredError", "message": "X-Apikey header is  
missing"}} (jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: hace una consulta cruda a la API de VirusTotal (recibirás un error de autenticación sin API key — regístrate gratis en [virustotal.com](https://www.virustotal.com) y consigue una, es una herramienta que usarás toda tu carrera).

Comando 2:

```
whois 8.8.8.8 | grep -iE "OrgName|Country|NetName"
```



```
missing"}} (jo4nlu♥parrot)-[~]  
[ ✓ ] $ whois 8.8.8.8 | grep -iE "OrgName|Country|NetName"  
NetName:      GOG  
OrgName:      Google LLC  
Country:      US  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: obtiene atribución básica de una IP — organización propietaria y país — el primer paso de cualquier triage de una IP sospechosa en tus logs.

Comando 3:

dig +short google.com A google.com AAAA google.com MX

```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ dig +short google.com A google.com AAAA google.com MX  
108.177.123.100  
108.177.123.113  
108.177.123.139  
108.177.123.101  
108.177.123.102  
108.177.123.138  
2800:3f0:4003:c0f::66  
2800:3f0:4003:c0f::64  
2800:3f0:4003:c0f::8b  
2800:3f0:4003:c0f::65  
10 smtp.google.com.  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: practica consultas DNS específicas por tipo de registro — habilidad esencial para investigar infraestructura de dominios sospechosos, como verificar si un dominio recién registrado tiene registros MX (capacidad de enviar phishing).

Tema 9: Ciclo de Vida de Respuesta a Incidentes (NIST SP 800-61)

Concepto

4 fases: **Preparación** (playbooks, entrenamiento, antes del incidente), **Detección y Análisis** (identificar y determinar alcance), **Contención/Erradicación/Recuperación** (detener propagación, eliminar causa raíz, restaurar operaciones), **Actividad Post-Incidente** (lecciones aprendidas, mejora de detecciones).

Caso real: Colonial Pipeline (2021) — un ransomware (DarkSide) entró vía una sola contraseña VPN sin MFA. La falta de segmentación clara entre TI y OT forzó una contención extremadamente drástica: apagar todo el oleoducto, causando escasez de combustible regional.

Laboratorio

Comando 1:

```
sudo ss -K dst 0.0.0.0/0 2>/dev/null
```

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ sudo ss -K dst 0.0.0.0/0 2>/dev/null
[sudo] contraseña para jo4nlu:
Netid          State          Recv-Q         Send-Q         Local Address:
Port          Peer Address:Port
u_str          ESTAB          0              0              *
435162         * 435161
u_str          ESTAB          0              0              /run/systemd/journal/stdout
203558         * 205940
u_dgr          ESTAB          0              0              *
22792         * 14055
u_str          ESTAB          0              0              *
1883429        * 1880528
u_str          ESTAB          0              0              /run/user/1000/pipewire-0
279902         * 214221
```

Qué hace: introduce ss -K, la herramienta real para matar conexiones de red activas de forma quirúrgica durante contención (úsalo filtrando por IP/puerto específico en un caso real, nunca en modo masivo salvo emergencia extrema).

Comando 2:

`sudo iptables -I INPUT -s 203.0.113.50 -j DROP && sudo iptables -L INPUT -n --line-numbers`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ sudo iptables -I INPUT -s 203.0.113.50 -j DROP && sudo iptables -L I
NPUT -n --line-numbers
[sudo] contraseña para jo4nlu:
Chain INPUT (policy DROP)
num  target      prot opt source                destination
1    DROP        all  --  203.0.113.50          0.0.0.0/0
2    LIBVIRT_INP all  --  0.0.0.0/0             0.0.0.0/0
3    ts-input    all  --  0.0.0.0/0             0.0.0.0/0
4    ufw-before-logging-input all  --  0.0.0.0/0             0.0.0.0/0

5    ufw-before-input all  --  0.0.0.0/0             0.0.0.0/0
6    ufw-after-input all  --  0.0.0.0/0             0.0.0.0/0
7    ufw-after-logging-input all  --  0.0.0.0/0             0.0.0.0/0
8    ufw-reject-input all  --  0.0.0.0/0             0.0.0.0/0
9    ufw-track-input all  --  0.0.0.0/0             0.0.0.0/0
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: simula contención mediante firewall — bloquear una IP maliciosa específica a nivel de host — la acción de contención más común y menos disruptiva que existe.

Comando 3:

`sudo tar -czf /tmp/evidencia_$(date +%Y%m%d_%H%M%S).tar.gz /var/log/auth.log /var/log/syslog 2>/dev/null && ls -lh /tmp/evidencia_*.tar.gz`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ sudo tar -czf /tmp/evidencia_$(date +%Y%m%d_%H%M%S).tar.gz /var/log/
auth.log /var/log/syslog 2>/dev/null && ls -lh /tmp/evidencia_*.tar.gz
-rw-r--r-- 1 root root 138 jul 19 22:13 /tmp/evidencia_20260719_221307.tar.gz
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: practica preservación de evidencia — empaquetar logs relevantes con timestamp antes de tocar más el sistema — un hábito no negociable, porque una vez que reinicias un servicio esa evidencia volátil se pierde para siempre.

Tema 10: Windows Event Logs — Cimientos del SOC en Entornos Windows

Concepto

Windows almacena eventos en archivos **.evtx** organizados en canales: Security, System, Application, y canales adicionales como Sysmon.

Event IDs clave: **4624** (logon exitoso), **4625** (logon fallido), **4688** (creación de proceso), **4672** (privilegios especiales asignados), **4720** (cuenta creada).

Patrón de investigación: movimiento lateral tipo Pass-the-Hash = 4624 Logon Type 3 anómalo + 4688 con herramientas de reconocimiento (net.exe, whoami.exe).

Caso real: Detección de **Mimikatz** vía Sysmon Event ID 10 (acceso a proceso) apuntando a LSASS.exe — patrón que cualquier SOC maduro alerta como crítico.

Laboratorio

Comando 1:

apt search evtx 2>/dev/null | grep -i evtx

```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ apt search evtx 2>/dev/null | grep -i evtx  
libevtx-dev/parrot 20240504-1+b1 amd64  
libevtx-utils/parrot 20240504-1+b1 amd64  
libevtxlt64/parrot 20240504-1+b1 amd64  
python3-evtx/parrot,parrot 0.7.4-1.1 all  
python3-libevtx/parrot 20240504-1+b1 amd64  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: busca en los repositorios de Parrot herramientas como evtx_dump o libevtx-utils, que permiten parsear archivos .evtx reales en Linux (Parrot, al ser una distro de seguridad ofensiva/forense, suele traerlas o permitir instalarlas fácilmente).

Comando 2:

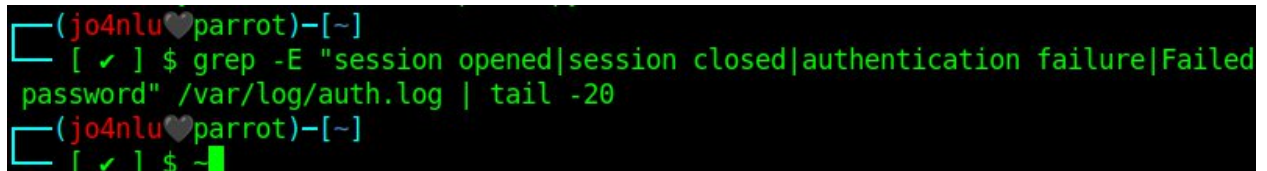
pip3 list 2>/dev/null | grep -i evtx || pip3 install python-evtx --break-system-packages

```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ pip3 list 2>/dev/null | grep -i evtx || pip3 install python-evtx --break-system-packages  
Defaulting to user installation because normal site-packages is not writeable  
Collecting python-evtx  
  Downloading python_evtx-0.8.1-py3-none-any.whl.metadata (6.0 kB)  
Collecting hexdump>=3.3 (from python-evtx)  
  Downloading hexdump-3.3.zip (12 kB)  
  Preparing metadata (setup.py) ... done  
Downloading python_evtx-0.8.1-py3-none-any.whl (26 kB)  
Building wheels for collected packages: hexdump  
  DEPRECATION: Building 'hexdump' using the legacy setup.py bdist_wheel mechanism, which will be removed in a future version. pip 25.3 will enforce this behavior change. A possible replacement is to use the standardized build interface by setting the '--use-pep517' option, (possibly combined with '--no-build-isolation'), or adding a 'pyproject.toml' file to the source tree of 'hexdump'. Discussion can be found at https://github.com/pypa/pip/issues/6334  
  Building wheel for hexdump (setup.py) ... done  
Created wheel for hexdump: filename=hexdump-3.3-py3-none-any.whl size=8934 sha256=564ccc8d8a6e422953d4b13a6063964db467f1645e8816b7382b27758c8ae040  
Stored in directory: /home/jo4nlu/.cache/pip/wheels/81/1e/a4/a5d574c8c252ae76a
```

Qué hace: verifica o instala python-evtx, una librería que usarás para analizar programáticamente logs de Windows exportados, cuando lleguemos a laboratorios con muestras reales.

Comando 3:

```
grep -E "session opened|session closed|authentication failure|Failed password"  
/var/log/auth.log | tail -20
```



```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ grep -E "session opened|session closed|authentication failure|Failed  
password" /var/log/auth.log | tail -20  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ ~
```

Qué hace: hace la traducción conceptual directa — en Linux, “session opened/closed” y “authentication failure” en auth.log son el equivalente funcional a los Event ID 4624/4625/4634 de Windows: mismo concepto (éxito/fallo de autenticación), sintaxis distinta.

Cierre de Fase 1

Con estos 10 temas se completan los **fundamentos de SOC**: estructura organizacional, logs, redes, sistemas operativos, frameworks de inteligencia de amenazas (Kill Chain / ATT&CK), malware, criptografía aplicada, threat intelligence, respuesta a incidentes y logging en Windows.

Próxima fase: Fase 2 — Herramientas SIEM/EDR en profundidad (Wazuh, ELK Stack), construcción de un laboratorio de detección real, y threat hunting activo.